

Article

Study on the Multi-Equipment Integrated Scheduling Problem of a U-Shaped Automated Container Terminal Based on Graph Neural Network and Deep Reinforcement Learning

Qinglei Zhang ¹, Yi Zhu ², Jiyun Qin ¹, Jianguo Duan ¹, Ying Zhou ^{1,*}, Huaixia Shi ¹ and Liang Nie ¹

¹ China Institute of FTZ Supply Chain, Shanghai Maritime University, Shanghai 201306, China; qlzhang@shmtu.edu.cn (Q.Z.); jyyqin@shmtu.edu.cn (J.Q.); jgduan@shmtu.edu.cn (J.D.); shihx@sdju.edu.cn (H.S.); 202240510015@stu.shmtu.edu.cn (L.N.)

² Institute of Logistics Science and Engineering, Shanghai Maritime University, Shanghai 201306, China; zy1502416775@163.com

* Correspondence: zhouying@shmtu.edu.cn

Abstract: Intelligent Guided Vehicles (IGVs) in U-shaped automated container terminals (ACTs) have longer travel paths than those in conventional vertical layout ACTs, and their interactions with double trolley quay cranes (DTQCs) and double cantilever rail cranes (DCRCs) are more frequent and complex, so the scheduling strategy of a traditional ACT cannot easily be applied to a U-shaped ACT. With the aim of minimizing the maximum task completion times within a U-shaped ACT, this study investigates the integrated scheduling problem of DTQCs, IGVs and DCRCs under the hybrid “loading and unloading” mode, expresses the problem as a Markovian decision-making process, and establishes a disjunctive graph model. A deep reinforcement learning algorithm based on a graph neural network combined with a proximal policy optimization algorithm is proposed. To verify the superiority of the proposed models and algorithms, instances of different scales were stochastically generated to compare the proposed method with several heuristic algorithms. This study also analyses the idle time of the equipment under two loading and unloading modes, and the results show that the hybrid mode can enhance the operational effectiveness. of the U-shaped ACT.



Received: 23 December 2024

Revised: 12 January 2025

Accepted: 20 January 2025

Published: 22 January 2025

Citation: Zhang, Q.; Zhu, Y.; Qin, J.;

Duan, J.; Zhou, Y.; Shi, H.; Nie, L.

Study on the Multi-Equipment Integrated Scheduling Problem of a U-Shaped Automated Container Terminal Based on Graph Neural Network and Deep Reinforcement Learning. *J. Mar. Sci. Eng.* **2025**, *13*, 197. <https://doi.org/10.3390/jmse13020197>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: graph neural network; integrated scheduling; U-shaped automated container terminal; disjunctive graph model; loading and unloading hybrid mode

1. Introduction

Over the past few years, as economic globalization has advanced and shipping tonnage has significantly expanded, the role of container terminals in the transport industry has become increasingly important. Currently, the problem of high carbon emissions associated with port operations is receiving increasing attention. Equipment for handling containers at terminals contributes to 15% to 25% of the total operational expenses in terms of energy costs [1]. Traditional automated container terminals (ACTs) usually adopt a vertical layout, which uses end-of-line handling, i.e., automated guided vehicles and external collector trucks can only travel to the buffer zones on the sea and land sides of the yard. Meanwhile, non-cantilevered rail cranes carry the containers over long distances, a process which consumes considerable time and energy. The design of the newly developed U-shaped automated container terminal is illustrated in Figure 1; this design adopts side handling, and the Intelligent Guided Vehicle (IGV) adopts visual navigation instead of the traditional magnetic peg navigation. The IGVs have the ability to autonomously avoid obstacles, and

they can freely travel to and fro between the Double Trolley Quay Crane (DTQC) and the Double Cantilevered Railway Crane (DCRC). They can also travel to the position of the target cabin in the yard, which obviates the need for the DCRC to carry out long-distance movements or reversing. The travelling paths of the IGVs and external trucks (ETs) are separated, and the ETs travel in one direction along a U-shaped path to avoid path conflicts. This layout reduces the terminal's operating costs, improves efficiency and furnishes a direction for the restructuring of traditional container terminals.

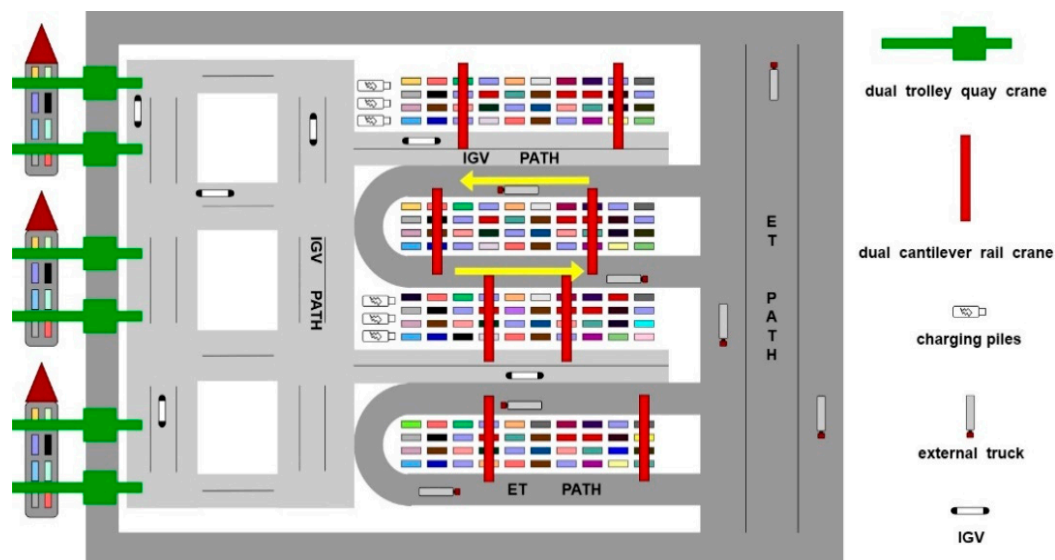


Figure 1. U-shaped automated container terminal layout.

This study makes the following contributions: (1) The integrated scheduling of DTQC, IGV and DCRC in U-shaped ACTs under the hybrid “loading and unloading” mode is studied (In the following text, it is referred to “the hybrid mode”), whereas few studies have paid attention to this problem in the past. (2) This paper innovatively proposes a scheduling framework based on graph neural network and deep reinforcement learning to solve the multi-device integrated scheduling problem of U-shaped ACTs.

With the aim of minimizing the maximum completion time of the U-shaped ACT and verifying the superiority of the proposed models and algorithms, this paper studied the integrated scheduling of DTQC, IGV, and DCRC, and proposed a scheduling framework to solve the problem.

The rest of the paper is organized as follows: Section 2 briefly reviews the related research on multi-device integrated scheduling of traditional automated container terminals and U-shaped automated container terminals, and deep reinforcement learning applied to scheduling problems. Section 3 describes the problem studied in this paper and proposes a scheduling framework based on graph neural networks and deep reinforcement learning. Section 4 introduces the algorithm, which includes disjunctive graph model construction, Markov decision process, parameterization strategy, and training algorithm. Section 5 validates the efficacy of the suggested model and algorithm via numerical experiments. Section 6 includes conclusions and future research directions.

2. Literature Review

Container transport tasks require multiple equipment resources to work together in an automated container terminal, and scheduling only one type of equipment individually can lead to sub-optimal solutions. Therefore, integrated scheduling of operational equipment in the ACTs has become one of the most effective ways to improve the operational

efficiency and equipment utilization of container terminals and reduce costs. A number of scholars have already contributed in this regard: Yang et al. [2] studied the integrated scheduling problem of quay cranes (QCs), automated guided vehicles (AGVs), automated rail mounted gantries (ARMGs) and AGV route planning, establishing a two-layer planning model to minimize the total completion time, and put forward a two-tiered genetic algorithm grounded in congestion avoidance principles to tackle the model. Luo et al. [3] integrated the scheduling of the components of the operational equipment of an ACT to create a MIP model incorporating loading components aimed at reducing vessel mooring duration, and examine how the loading equipment utilized influences the mooring and computational durations. Kong et al. [4] minimized the completion time of tandem quay crane unloading operations by considering the interaction between QC and AGV. Zhuang et al. [5] investigated the integrated scheduling problem of intelligent operational equipment for ACTs with the challenging features of bidirectional flow and finite buffer. Shouwen Ji et al. [6] considered both the integrated scheduling of operational equipment and the AGV path planning issue; they developed a two-layer planning model for this combinatorial optimization problem with the goal of reducing the total completion time. Wang et al. [7] investigated the integrated scheduling for container transport systems considering direct, buffer and mixed modes of transport, formulating it as a multi-layer vehicle path problem and connecting them through priority relations and transfer constraints. Duan et al. [8] proposed an integrated scheduling optimization model for AGVs and automated stacker cranes (ASCs) to address the problems of long round-trip time and high energy consumption of ASCs in a typical ACT, and analyzed the configuration of ACTs considering land-side buffers.

As an emerging port layout mode, the increased number of equipment interaction points in the U-shaped layout, as well as the horizontal transport equipment driving into the yard to carry out loading and unloading operations in a double-cycle mode, have greatly increased the complexity of and the need for integrated scheduling problems at the terminal, as compared with the conventional horizontal and vertical layout modes. With the aim of addressing these complex problems, some scholars began to turn their research to U-shaped automated container terminals. Xu et al. [9] proposed an integrated scheduling optimization model for container terminals based on mixed integer programming to analytically characterize the loading and unloading process of a U-shaped automated container terminal, which eliminates the waiting time, achieves temporal and spatial synchronization, and minimizes the completion time of all tasks during the interaction between AGVs and a double cantilever rail crane. Junjun Li et al. [10] proposed a scheduling framework featuring a hierarchical abstraction of scheduling entities, and proposed a combined dynamic and static scheduling strategy. Yongsheng Yang et al. [11] developed a detailed collaborative scheduling framework for multiple devices with U-shaped ACT based on four loading and unloading modes of DCRC with the objective of the shortest total running time. Xu et al. [12] developed a combined scheduling model for seamless coordination of QCs, AGVs, and DCRCs during the unloading process. This model aims to reduce the processing time for each task in a U-shaped automated container terminal and prevent concurrent conflicts among AGVs. Yaqian Niu et al. [13] investigated the ADCRCs, IGVs and ETs in the yard under multipoint loading and unloading mode in a U-shaped automated terminal, and considered the path planning problem for IGVs leaving the QC to reach the yard for cooperative scheduling. Liu W et al. [14] investigated a collaborative scheduling framework for yard cranes and horizontal transport devices based on the layout of a U-shaped yard and determined the spatio-temporal trajectory of the AGVs with collision avoidance in mind. Lei Cai et al. [15] proposed a MIP model to characterize the collaborative scheduling problem and designed a metric based on the entropy within the intricate network frame-

work to assess its resistance to cascading effects and scheduling robustness. From the above literature, it appears that there is a scarcity of research focusing on the collaborative scheduling of a U-shaped ACT's three equipment types: Quay Cranes, IGV, and Yard Crane. Most scholarly studies focus only on the scheduling of IGVs or only on the integration of scheduling between two devices.

Currently, most ACT scheduling problems are addressed through the application of conventional heuristic algorithms. In real port operating environments, diverse automation devices and other dynamical factors, such as equipment faults and the latency of the network, are often faced. This makes the overall scheduling vulnerable to random disturbances owing to the intricate and dynamic nature of the port environment and system information. Formulating the scheduling problem as a Markov decision-making process, the randomness of the system can be described by the probability of transition between states, so as to simulate random events. In addition, combined with reinforcement learning, Markov chains can learn optimal scheduling strategies. By interacting with the environment, the algorithm can constantly adjust the strategy to adapt to changes in the environment. Markov chains not only have advantages in scheduling problems, but also have achievements in other sequence decision-making problems such as container storage problems. Xin Jin et al. [16] designed a Markov Decision Process (MDP) model to simulate the container stacking process and enable the Deep Reinforcement Learning (DRL) agent to attempt to reduce the quantity of obstructions encountered during the retrieval of containers. In addition, ACTs are more efficient using a double-cycle loading and unloading mode than a single-cycle mode of unloading and then loading [17], but with this comes an increase in the complexity of the problem. Based on the above points, it is obvious that the traditional scheduling strategy is no longer applicable to U-shaped automated container terminals. General heuristic search algorithms are prone to fall into local optimums and have a high overhead of solution time, so it is essential to improve the efficiency of algorithmic solutions to meet the processing timeliness requirements of complex and changing port environments.

The application of deep learning algorithms to scheduling issues has emerged as a prominent trend in the research of combinatorial optimization. Deep learning algorithms learn the optimal strategy of conducting scheduling via simulation experiments or utilizing real data. Thus, they are capable of adjusting to various operational contexts and scheduling settings, while also possessing the ability to resist interference. Wang et al. [7] integrated the strengths of the real-time responsiveness and adaptability of Convolutional Neural Network (CNN) and Deep Deterministic Policy Gradient (DDPG) algorithms to dynamically modify AGV scheduling tactics according to the input spatial-temporal state data. Nitish Singh et al. [18] formulated the real-time scheduling decision-making problem for AGVs with battery constraints as an MDP, and proposed a solution based on deep reinforcement learning. Zhang L et al. [19] modelled the bi-objective joint optimization problem of AGV scheduling and battery replacement management as a Markov decision process, constructed an optimization architecture based on deep reinforcement learning, and created a new dueling deep double Q-network algorithm to maximize the long-term benefits of reducing delays in material handling and energy consumption optimization.

Graph Neural Networks (GNNs) can represent complex scheduling environments through a graph structure, which is very advantageous for expressing the complex relationships among jobs, processes, and machines in a scheduling problem; it can also capture the dependencies and resource constraints among jobs, enabling the model to better reflect the real-time status of jobs and make more accurate scheduling decisions [20]. Many scholars have used graph neural networks in shop floor scheduling problems. Xuan Jing et al. [21] developed a centralized learning decentralized execution (CLDE) multi-intelligence re-

inforcement learning scheduling framework based on Graph Convolutional Networks (GCNs) to address the Flexible Job Shop Scheduling Problem (FJSP). Chupeng Su et al. [22] proposed a framework based on GNN and DRL for solving Dynamic JSSP (DJSP) with machine failures and stochastic processing time. Lei K et al. [23] proposed an end-to-end deep reinforcement framework for automatically learning to use graph neural networks to solve the FJSP. Wen Song et al. [24] proposed a novel DRL method to learn high-quality PDR end-to-end to solve the flexible job shop scheduling problem, where operation selection and machine assignment are combined into a composite decision. Jiang-Ping Huang et al. [25] proposed an end-to-end DRL approach based on graph neural networks for solving the distributed job shop scheduling problem. Min Zhang et al. [26] presented an end-to-end approach for acquiring a high-quality scheduling policy tailored specifically for the dynamic flexible job shop scheduling problem with insufficient transportation resources. The port scheduling problem closely resembles the shop floor scheduling problem in that it is a sequential decision-making problem, and GNN can be used to solve the port scheduling problem. However, few scholars have investigated multi-equipment collaborative scheduling in ports through graph neural networks.

Upon reviewing the aforementioned literature, several research gaps remain, which are as follows:

- (1) At present, there are few studies on the multi-equipment integrated scheduling of U-shaped automated container terminals.
- (2) Few researchers have considered the interaction between DTQC, IGV, and DCRC in U-shaped automated container terminals.
- (3) The use of deep reinforcement learning to solve the integrated scheduling problem of U-shaped automated container terminals has not been studied.

In order to realize integrated scheduling between DTQCs, IGVs and DCRCs in U-shaped ACTs, this study investigates the integrated scheduling problem concerning the three devices (DTQC, IGV, and DCRC) in U-type ACTs under the mode. The problem is formulated as an MDP and a disjunctive graph model is developed to better display the characteristics of the multi-device integrated scheduling problem of U-shaped ACTs in order to quickly respond to the scheduling scheme. A graph deep reinforcement learning algorithm combining GNN and proximal policy optimization (PPO) algorithms is proposed as a framework for learning the U-shaped ACT multi-device integrated scheduling problem (UACT-ISP).

3. Problem Description and Scheduling Framework

In a U-shaped layout automated container terminal, the integrated scheduling problem of DTQC, IGV, and DCRC mainly concerns the connection of horizontal transportation of containers between the DTQC side and the DCRC side through IGV. At the same time as the IGV operation, DTQC and DCRC need to reasonably handle and allocate container handling tasks. Besides, with the aim of reducing the total time in port of ships, simultaneous loading and unloading of ships is usually adopted. This requires the IGV to alternately perform import and export container tasks.

In this section, a scheduling framework based on GNN and PPO algorithms is proposed to solve UACT-ISP, as shown in Figure 2. The framework is composed of three parts: scheduling environment, GNN-based scheduling agent and PPO algorithm training module.

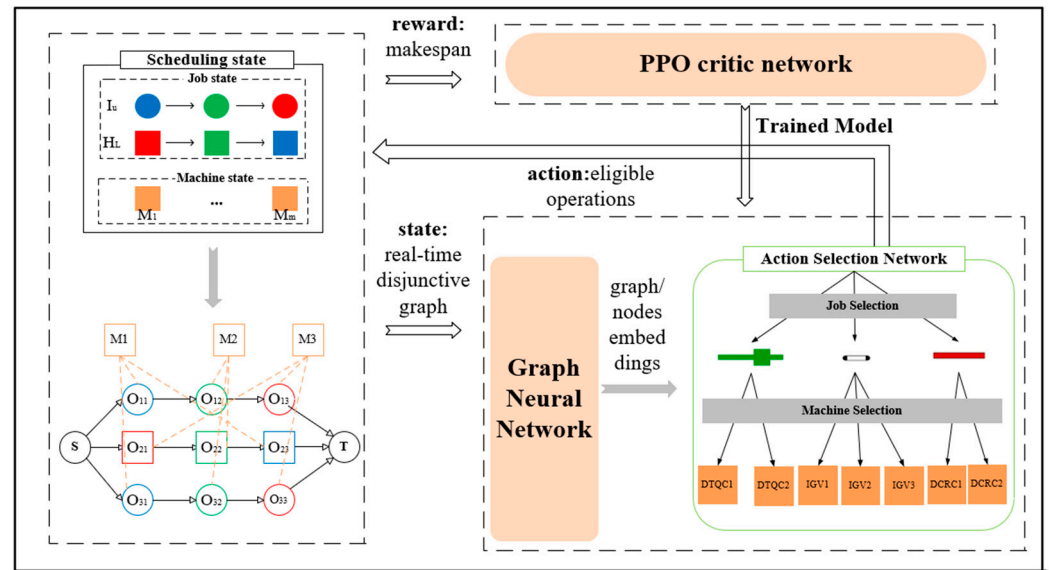


Figure 2. Schematic of the UACT-ISP framework based on graph neural networks and proximal policy optimization algorithms.

The framework transmits the current U-shaped ACT scheduling status to the scheduling agent using a disjunctive graph model representation. In each scheduling decision step, the real-time U-shaped ACT scheduling state contains import tasks and export tasks, each of which needs to go through three phases of DTQC, IGV, and DCRC operations, with a number of devices to choose from for each operation. The scheduling agent extracts the node embeddings and graph embeddings for this state using a GNN, parameterizes the two sub-policies using the action selection network of the PPO algorithm, maps the embeddings in order to select eligible phase operations including DTQC loading and unloading, IGV transport, and DCRC loading and unloading, and then selects idle devices for the operations. This procedure continues until every operation has been scheduled. Finally, the current strategy is evaluated and updated by the critic network of the PPO algorithm.

4. Solution Approach

In this study, solving the UACT-ISP is considered as a sequential decision-making process that iteratively takes scheduling actions and assigns the task's operations to the devices of the current process stage in each state until all the task's operations have been executed. In this section, Section 4.1 establishes the analytic graph model of UACT-ISP in the hybrid mode. Section 4.2 describes the analytic graph model-based UACT-ISP as an MDP, defining the states, actions, transition and reward functions of the analytic graph model of the UACT-ISP. Section 4.3, using the graph isomorphism network, devises two sub-policies, one for selecting the job operations and the other for selecting a device for the selected job action. Section 4.4 uses the PPO algorithm to train the policy network, consisting of two parallel actor networks and a criterion network. Section 4.5 describes the algorithm's comparison criterion.

4.1. UACT-ISP Disjunctive Graph Model

The disjunctive graph model, or DG model, represents the process of a scheduling problem as a directed graph $G = (O, C, D)$, where O is the set of all nodes of this directed graph (including virtual start and end nodes), each representing an operation; C is the set of all connecting arcs, a connecting arc is an arc between two adjacent operations on the same workpiece in a directed graph G , and the direction of the connecting arc represents the sequence of operations; D is the set of all disjunction arcs, which are arcs connecting two

consecutive job tasks for each device, and the direction of the disjunction arcs represents the sequence of job tasks performed by a device.

As the IGVs in the U-shaped ACT can transport containers directly to the corresponding storage bays in the yard, there is no need to transport containers to the yard via AGV mates or buffer zones, as is the case in conventional layouts. Therefore, in constructing the scheduling model, the operation process of a U-shaped ACT is abstracted into a three-phase hybrid flow shop scheduling process. Each container is regarded as a workpiece, and its operation at each stage is regarded as a process. Each container has to go through three processes, namely double trolley quay crane loading and unloading, IGV horizontal transport, and double cantilever rail crane handling, and there are several parallel devices to be allocated for each process. Figure 3a shows the disjunctive graph model of a UACT-ISP instance which contains all possible solutions, Figure 3b is a solution to Figure 3a, and Figure 4 shows the gantt chart representation of the solution in Figure 3b. For a U-shaped ACT scheduling problem in the hybrid mode, there are n import and export container tasks $T = \{T_i \mid i = 1, 2, 3, \dots, U + L\}$ processed on the devices $M = \{M_k \mid k = 1, 2, 3, \dots, m\}$, which is divided into the import task $IU = \{I_u \mid u = 1, 2, \dots, U\}$ and the export task $HL = \{H_o \mid o = 1, 2, \dots, L\}$. Each job task has v successive executions of a particular sequence of operations O_{ij} , where $O^{YC} = \{O_i^{YC} \mid i \in T_i\}$ is the set of dual cantilever rail crane operation tasks, $O^{IV} = \{O_i^{IV} \mid i \in T_i\}$ is the set of IGV horizontal transport operation tasks, $O^{QC} = \{O_i^{QC} \mid i \in T_i\}$ is the set of dual trolley quay crane operation tasks, and $O^0 = \{O_i^0 \mid i \in T_i\}$ is the set of virtual processes, so all the job task sets are $O_{ij} = \{O_{ij} \mid i \in T_i, j \in V_j\} = O^{YC} \cup O^{IV} \cup O^{QC} \cup O^0$. The import task has to satisfy a predefined processing order of $O_i^{QC} \rightarrow O_i^{IV} \rightarrow O_i^{YC}$, and the export task is the opposite $O_i^{YC} \rightarrow O_i^{IV} \rightarrow O_i^{QC}$. Each operation has a start time of S_{ij} , an operating time of P_{ij} , and each node, i.e., each operation, has a number of next actions waiting to be selected. In short, solving the U-shaped ACT multi-device integrated scheduling problem means determining a scheduling action for each operational task O_{ijk} that minimizes the total completion time.

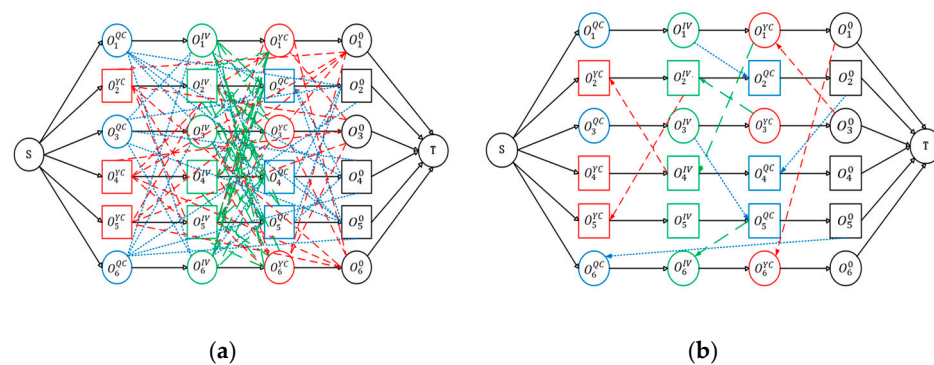


Figure 3. Disjunctive graph model for a concrete instance of a U-shaped ACT multi-device integrated scheduling problem. (a) A disjunctive graph with all feasible solutions; (b) an example of a feasible solution.

4.2. Markov Decision Process

The essential phase in solving the scheduling problem with deep learning is to reformulate it as a Markov Decision Process (MDP), specifying states, actions, state transitions, and reward functions. In this study, we take the real-time DG of the U-shaped ACT multi-device integrated scheduling problem as the state, the GNN as the state feature extractor, and formulate the following MDP model to solve it:

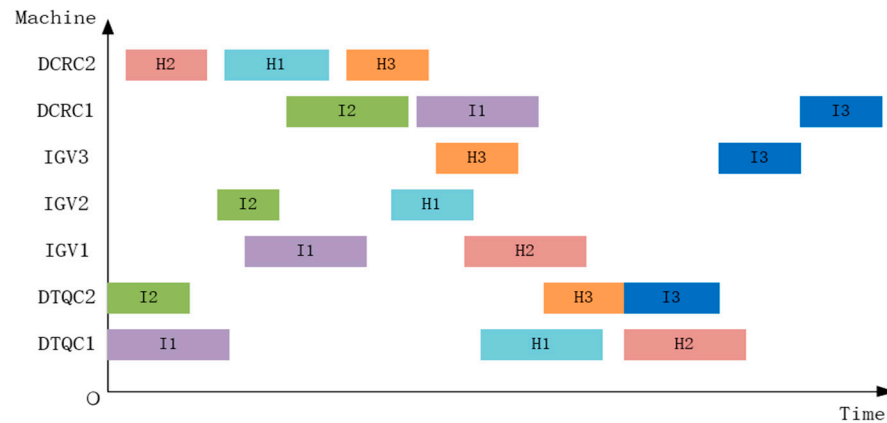


Figure 4. Gantt chart representation of Figure 3b.

States: The local states S_t^0 and S_t^m at timestep t constitute the global state S_t . The local state S_t^0 is a disjunctive graph $G_t = (O, C \cup D_t^0, D_t)$ where D_t^0 is the disjunction arcs of the identified directions up to timestep t (i.e., the actions represented in the disjunctive graph), and $D_t \subseteq D$ is the set of remaining disjunction arcs. The local state S_t^m consists of all device states, each containing two dimensional properties $[T_t(M_k), P_{ijk}]$, $k \in \{1, \dots, m\}$, where $T_t(M_k)$ is the completion time of device k before timestep t . In addition, P_{ijk} denotes the operation time of operation O_{ij} on device k when device k is available, otherwise it denotes the average processing time $\frac{1}{|K|} \sum_k P_{ijk}$, $k \in K$, where K is a set of available devices. Each node $O_{ij} \in O$ contains two features, $[LB_t(O_{ij}), I_t(O_{ij})]$, $i \in \{1, 2, \dots, n\}$, $j \in \{1, 2, \dots, v\}$, where $LB_t(O_{ij})$ denotes the finish time of the scheduled operation O_{ij} or the estimated shortest finish time of the unscheduled operation O_{ij} : $LB_t(O_{ij-1}) + \min(P_{ijk}, k \in M)$. $I_t(O_{ij})$ is a one-dimensional feature which is equal to 1 when the operation O_{ij} has been scheduled, and 0 otherwise.

Actions: An action executed at timestep t encompasses a job operation action $a_o \in A_0$ and a device action $a_m \in A_m$. A_0 is the collection of permissible operations and A_m is the collection of devices with compatibility for operations A_0 . The job operation action space a_o is contingent upon the intersection of unscheduled jobs, while the equipment action space a_m is contingent upon the quantity of compatible devices for the operation which has been selected.

Transition: At timestep t , the agent takes the sample for action a_t and chooses a device for that action. The orientation of the disjunction arc is subsequently modified in accordance with the present task action choice and the chosen device. A novel disjunctive graph emerges as the local state denoted by S_{t+1}^0 . The device finish time $T_{t+1}(M_k)$ and the operation time P_{ijk} of a_t are then updated as the local state S_{t+1}^m .

Reward: The reward is defined as the difference between the completion times of partial schedules S_t and S_{t+1} , i.e., $r(S_t, a_t, S_{t+1}) = C_{max}(S_t) - C_{max}(S_{t+1})$. When the discount factor $\gamma = 1$, the cumulative reward of a solution set is $G = \sum_{t=0}^{|O|} r(S_t, a_t, S_{t+1}) = C_{max}(S_0) - C_{max}(S_{t+1})$. For a particular problem instance, $C_{max}(S_0)$ is a constant, which means that minimizing C_{max} and maximizing G are equivalent.

Policy: The strategy $\pi_\theta(a_t | S_t)$ defines a probability distribution over the set of actions a_t for each state S_t . It is divided into two sub-strategies, $\pi_{\theta_O}(O_{ij} | S_t)$ and $\pi_{\theta_M}(M_k | S_t)$, where θ_O and θ_M are the parameters of the job selection strategy and the equipment selection strategy, respectively, and $\theta = [\theta_O, \theta_M]$ is the set of all parameters.

4.3. Parameterizing Two Sub-Policies

The above disjunctive graph of UACT-ISP provides a complete state view of scheduling, which contains the priority constraints, the processing order on each device, the set

of parallel devices for each operation, and the processing time of the parallel devices for each operation. To obtain effective scheduling performance, it is essential to retrieve all the state details contained within the disjunctive graph. In this study, we encode disjunctive graphs using Graph Isomorphism Network (GIN), a popular GNN variant with strong discriminative capabilities and powerful isomorphism tests [27].

4.3.1. Graph Embedding (Job Operation Encoder)

At timestep t , the local state S_t^o is defined by the disjunctive graph $G_t = (O, C \cup D_t^0, D_t)$, where each node is encoded by an L -layer GIN, each layer of which is shown in Equation (1). $h_j^{(l)}$ is the embedding of node j in the l -th layer of GIN, h_j^0 is the initial feature of node j , $MLP_{\theta_l}^{(l)}$ is the multi-layer perceptron (MLP) for the l -th layer and θ_l are the parameters of this layer [28], $\epsilon^{(l)}$ is a learnable parameter, and $N(j)$ is the set of neighborhood nodes of node j .

$$h_j^{(l)} = MLP_{\theta_l}^{(l)} \left((1 + \epsilon^{(l)}) \cdot h_j^{(l-1)} + \sum_{u \in N(j)} h_u^{(l-1)} \right). \quad (1)$$

After L iterations, the global representation of the entire disjunctive graph can be extracted using the average pooling function, as shown in Equation (2):

$$h_g = 1/|V| \sum_{j \in V} h_j^{(L)}. \quad (2)$$

GIN was originally proposed for undirected graphs in the literature [27]. However, in UACT-ISP, the disjunctive graph G_t associated with each state S_t is a hybrid graph with directed arcs, which describes key characteristics such as priority constraints and sequences of operations on the device. Therefore, we need to promote GIN to support disjunctive graphs. A simple and straightforward way is to connect every two operation nodes of the same type by directed arcs, thus obtaining an all-directed disjunctive graph, as shown in Figure 5a. Then, the set of neighborhood nodes of node v in Equation (1) is denoted as $N(v) = \{u \mid (u, v) \in E(G_t)\}$, where $E(-)$ is the set of arcs of the graph, i.e., $N(v)$ contains all the incoming neighbors of v ; this allows GIN to operate on G_t . Figure 5b shows an example of this “arc cancellation strategy”, which removes the directed arcs (O_{33}, O_{21}) , (O_{22}, O_{13}) , (O_{33}, O_{13}) , (O_{13}, O_{33}) , and obtains the direction of the disjunction arcs, i.e., the order of operation of the DCRC is $O_{13} \rightarrow O_{21} \rightarrow O_{33}$.

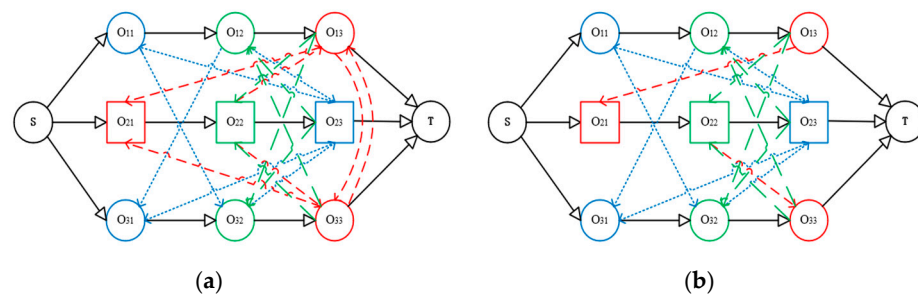


Figure 5. Fully directed disjunctive graph (a) and “arc cancellation strategy” (b) of UACT-ISP.

However, this approach leads to a dense disjunction graph that cannot be efficiently processed by GIN. In this study, we adopt the “add-arc strategy” [29], which ignores the initial state of undirected disjunction arcs, so that $\overline{G}_t = (O, C \cup D_t^0)$ is approximated by $G_t = (O, C \cup D_t^0, D_t)$, and as the scheduling process proceeds, D_t becomes larger because more directed arcs are added to it. Figure 6 illustrates this “add-arc strategy”, where directed arcs (O_{22}, O_{33}) are added to the disjunctive graph after the current operation.

Obviously, this strategy makes the state graph representation of UACT-ISP more sparse, which can reduce the computational complexity.

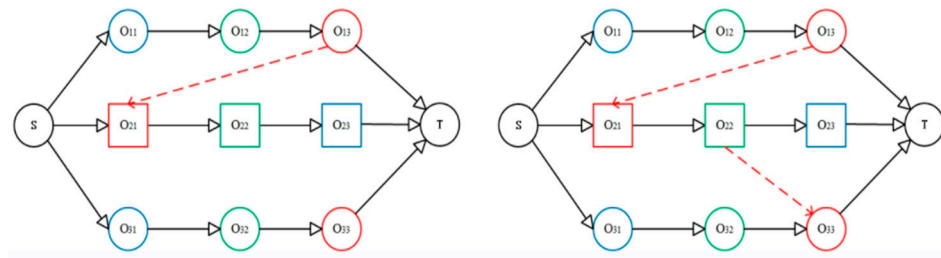


Figure 6. The “arc-adding strategy” of the disjunctive graph model of UACT-ISP.

4.3.2. Node Embedding (Device Encoder)

In the state information of a device, no diagrammatic framework exists where every node symbolizes the key attributes of each apparatus. Every node lacks connections to other vertices via either oriented or non-oriented edges. Thus, a fully connected layer is used to encode the local state S^m . At time step t , the main characteristics of each device node in the local state S_t^m are defined as a two-dimensional vector $[T_t(M_k), P_{ijk}]$, $k \in \{1, \dots, m\}$, as previously described. The outputs of each node (device) consist of the embedding vector h_k^t and the pooling vector u^t .

4.3.3. Action Selection (Decoders)

The completed schedules are constructed sequentially by decoding at each timestep $t \in \{1, \dots, |O|\}$, where $|O|$ is the total number of operations in the UACT-ISP instances. At each timestep t , the job decoder and the device decoder select a job operation action a_t^o , and a device action a_t^m , respectively, based on the local states embedded by encoder, S_t^o and S_t^m , to form a complete action, $a_t = (a_t^o, a_t^m)$.

The pair of decoders are founded upon MLP layers and boast congruent network configurations, but refrain from parameter sharing. During the decoding process, each decoder calculates a probability distribution over either the job operation action space or the device action space. Initially, the two decoders are utilized to generate a job operation action score $C_{t,j}^o$ and a device action score $C_{t,k}^m$, computed as Equations (3) and (4), where $\bullet \parallel \bullet$ is the concatenation operation.

$$C_{t,j}^o = \text{MLP}_{\theta_{\text{To}}} \left(h_j^{(L)} \parallel |h_g| \parallel |u^t| \right), j \in \{1, 2, \dots, v\}. \quad (3)$$

$$C_{t,k}^m = \text{MLP}_{\theta_{\text{Tm}}} \left(h_k^t \parallel |h_g| \parallel |u^t| \right), k \in \{1, 2, \dots, m\}. \quad (4)$$

Then, a masking scheme similar to [30] is used to avoid (1) scheduling operations that are already arranged or those that breach precedence constraints and (2) choosing devices incapable of handling an operation. To elaborate, operations that have been scheduled or contravene the precedence constraint are concealed by setting their score $C_{t,j}^o$ and $C_{t,k}^m$ to $-\infty$. Likewise, we conceal devices that are not compatible with the chosen operation at timestep t .

We standardize the scores for job operation actions and device actions by a Softmax function as shown in Equations (5) and (6):

$$P_i(a_t^o) = \frac{e^{C_{t,i}^o}}{\sum_j e^{C_{t,j}^o}}. \quad (5)$$

$$P_f(a_t^m) = \frac{e^{C_{t,f}^m}}{\sum_k e^{C_{t,k}^m}}. \quad (6)$$

Ultimately, a strategy of either random sampling or greedy decoding is employed to forecast a_t^o and a_t^m based on $P_i(a_t^o)$ and $P_f(a_t^m)$. In the process of training, the random sampling strategy samples the nodes to be selected according to the probability distribution and constructs the effective solution, which helps to explore the environment for better model performance. In the testing phase, the greedy decoding strategy greedily selects the most possible node at each timestep.

4.4. Training Algorithm

Proximal Policy Optimization (PPO), an actor–critic architecture, is a promising variant of the policy gradient approach that improves network training efficiency [31]. In this study, we use PPO as a policy optimization method to build two parallel actor networks that learn two sub-policies to select job operation actions and select a device action a_m for the chosen job operation action a_o at each timestep. With the aim of generating the policy $\pi_{\theta_O}(O_{ij} | S_t)$ for discrete job operation actions and the policy $\pi_{\theta_M}(M_k | S_t)$ for discrete device actions, the two actor networks of the PPO output probability distributions for the job operation space and device action space, respectively. Finally, the job action and the device action are selected by a decoding strategy (random sampling strategy or greedy decoding strategy, as described previously) on the generated probability distribution. The critic network V_φ uses a MLP_{θ_V} that takes as input the global state h_g at S_t and outputs a scalar to estimate the cumulative reward at S_t . Furthermore, this research employs the Adam optimizer to dynamically adjust the learning rate for each network parameter, enhancing convergence speed and training stability. Algorithm 1 provides the specific training procedure of the PPO algorithm for the model.

Algorithm 1: Training method: PPO

Input: number of iterations : K_i ; number of training steps : N ; update epoch : K_e ; discount factor: γ ; batch size: X ; actor network: π_θ and behaviour actor network: $\pi_{\theta_{old}}$; critic network: V_φ ; policy loss coefficient: C_p ; value function loss coefficient: C_v ; entropy loss coefficient: C_e ; clipping ratio: ϵ .

Initialization: π_θ , $\pi_{\theta_{old}}$ and V_φ ;

For $n = 1, 2, \dots, N$ **do**

Sampling X UACT-ISP instances;

for $x = 1, \dots, X$ **do**

for $t = 0, 1, 2, \dots$ **do**

Sample $a_{x,t}$ based on $\pi_{\theta_{old}}(a_{x,t} | S_{x,t})$;

Receive reward $r_{x,t}$ and next state $S_{x,t+1}$;

$\hat{A}_{x,t} = \sum_0^t \gamma^t r_{x,t} - V_\varphi(S_{x,t})$, $r_{x,t}(\theta) = \frac{\pi_\theta(a_{x,t} | S_{x,t})}{\pi_{\theta_{old}}(a_{x,t} | S_{x,t})}$;

if $S_{x,t}$ is terminal **then**

break

end

end

$L_x^{CLIP}(\theta) = \sum_0^t \min(r_{x,t}(\theta) \hat{A}_{x,t}, \text{clip}(r_{x,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{x,t})$;

$L_x^{VF}(\varphi) = \sum_0^t (V_\varphi(S_{x,t}) - \hat{A}_{x,t})^2$;

$L_x^S(\theta) = \sum_0^t S(\pi_\theta(a_{x,t} | S_{x,t}))$, where $S(\cdot)$ is entropy;

Aggregate losses : $L_x(\theta, \varphi) = C_p L_x^{CLIP}(\theta) - C_v L_x^{VF}(\varphi) - C_e L_x^S(\theta)$;

end

for PPO update epoch = $1, 2, \dots, K_e$ **do**

Update θ, φ with cumulative loss by Adam optimizer:

$\theta, \varphi = \text{argmax}(\sum_{x=1}^X L_x(\theta, \varphi))$

end

$\theta_{old} \leftarrow \theta$

End

Output: Trained parameter set $\theta = [\theta_O, \theta_M]$.

4.5. Algorithm Comparison Guidelines

In this study, Equation (7) is used to calculate the computational performance gap between the heuristic algorithm and the proposed method on UACT-ISP instances:

$$Gap_{ours-method} = \frac{C_{max_{method}} - C_{max_{ours}}}{C_{max_{ours}}} \times 100\%, \quad (7)$$

where $C_{max_{ours}}$ represents the objective function value of our method, and $C_{max_{method}}$ represents the objective function value of other compared algorithms.

5. Numerical Experiments

In this section, Section 5.1 sets the parameters used to stochastically generate UACT-ISP instances and the hyperparameters of the algorithms. Section 5.2 describes the dataset. Section 5.3 shows the computational results of the different algorithms on stochastically generated instances of different sizes and compares them with the methods proposed in this study. Section 5.4 presents a comparative analysis of the performance of the “unloading before loading” mode and the hybrid mode in terms of makespan and performance of device spare time.

The experiments were conducted in the following hardware environment: an NVIDIA Quadro RTX 8000 GPU, an Intel Xeon Gold 6246R CPU, and 512 GB of RAM. The software environment consisted of Python 3.10 and PyTorch 2.1.

5.1. Parameter Settings

5.1.1. Problem Parameters

The number of container tasks is 3–100, of which the number of import tasks is half of the total number of tasks, rounded up if it is not an integer; the number of IGVs is 2–15, the number of DTQCs is 2–8, and the number of DCRCs is 4–10. As mentioned earlier, both the import task and the export task are subject to three phases of work, namely the DTQC loading and unloading phase, the IGV horizontal transport phase, and the DCRC loading and unloading phase. The data distribution of each parameter at each stage and the terminal layout parameters are shown in Table 1. The time for the main trolley of the DTQC to take down the import task from the ship or put the export task on the transfer platform obeys a uniform distribution of (20, 30) s. The time for the portal trolley of the DTQC to put the import task on the IGV or to take down the export task from the IGV is 20 s, and the capacity of the transfer platform is constrained to be three [2]. The length of the IGV is 13 m, the safety distance between two IGVs is 5 m, the speed of the IGVs is 5 m/s for both empty and loaded IGVs, and the density of IGVs entering the yard is constrained to be a minimum of three and a maximum of six. The time taken by the DCRC to move the containers from the IGVs to the designated storage point or to move the containers from the storage point to the IGVs obeys a uniform distribution of (30 s, 50 s) [9]. The horizontal transport area is 300 m long and 120 m wide, the container bays in the yard obey a uniform distribution of (1, 20), and the length of a bay in the yard is 15 m.

5.1.2. Hyperparameters

In this study, fixed training parameters are used to train GINs of different sizes, as shown in Table 2. To strike a balance between solution quality and efficiency, this study sets the number of GIN layers and fully connected layers in the MLP to two, the hidden dim to 128, and the learning rate to 0.001 in accordance with the experience of previous research [22].

Table 1. The parameter settings of UACT-ISP instances.

	Parameter	Distribution	Unit	Reference
DTQC	main trolley work time	U (20, 30)	s	[31]
	portal trolley work time	20	s	[31]
	the capacity constraint of transfer platform	3	—	[31]
IGV	length	13	m	[9]
	save distance	5	m	[9]
	velocity	5	m/s	[9]
DCRC	loading and unloading time	U (30, 50)	s	[9]
LAYOUT	the length of the horizontal transportation area	300	m	—
	the width of the horizontal transportation area	120	m	—
	the bays of containers in the yard	U (1, 20)	—	—
	the length of a bay	15	m	—

Table 2. The hyperparameters of the training method.

Description	Notation	Value
optimizer	Adam	—
discount factor	γ	1
clip ratio	ϵ	0.2
value function loss coefficient	C_v	1
policy loss coefficient	C_p	2
entropy loss coefficient	C_e	0.01
update epoch	K_e	350
iteration	K_i	50
training steps	N	100

5.2. Dataset

In this study, various scales of UACT-ISP instances are stochastically generated based on the parameter settings outlined in Table 1, as shown in Table 3, where the scale is expressed as “number of containers-number of DTQCs-number of IGVs-number of DCRCs”. To minimize training time and evaluate the generalizability of the proposed method, small-scale stochastically generated instances (i.e., Nos. 1–8) are used for training and validation, and then large-scale stochastically generated instances (i.e., Nos. 9–12) are used as the test set. Table 4 shows an example of a UACT-ISP instance with a minimum scale of 3-2-2-4, where each task contains three operations and “-” indicates that the device cannot process an operation.

Table 3. The UACT-ISP instances generated stochastically.

Number	Scale (Tasks-DTQCs-IGVs-DCRCs)
1	3-2-2-4
2	5-2-3-4
3	10-2-5-4
4	10-4-5-4
5	20-2-5-4
6	20-4-5-8
7	30-2-10-4
8	30-4-10-8
9	50-4-10-4
10	50-8-15-10
11	100-4-10-4
12	100-8-15-10

Table 4. An example 3-2-2-4 UACT-ISP instance.

P_{ijk}	O_1^{QC}	I_1 O_1^{IV}	O_1^{YC}	O_2^{YC}	H_1 O_2^{IV}	O_2^{QC}	O_3^{QC}	I_2 O_3^{IV}	O_3^{YC}
$DTQC_1$	46.1	—	—	—	—	45.3	49.4	—	—
$DTQC_2$	46.9	—	—	—	—	43.6	40.3	—	—
IGV_1	—	57.5	—	—	64.3	—	—	52.8	—
IGV_2	—	63.8	—	—	67.9	—	—	64.6	—
$DCRC_1$	—	—	44.4	49.6	—	—	—	—	39.4
$DCRC_2$	—	—	35.7	42.2	—	—	—	—	37.8
$DCRC_3$	—	—	46.1	33.8	—	—	—	—	45.1
$DCRC_4$	—	—	40.8	40.1	—	—	—	—	47.9

In the training set, for problems of different scales, this study adopts different batch sizes, with each batch containing 32 UACT-ISP instances of scale 3-2-2-4 and 5-2-3-4, 16 UACT-ISP instances of scale 10-2-5-4 and 10-4-5-4, eight UACT-ISP instances of scale 20-2-5-4 and 20-4-5-8, and four UACT-ISP instances of scale 30-2-10-4 and 30-4-10-8.

5.3. Computational Results of Different Algorithms for Stochastically Generated Instances

The proposed method was first evaluated on stochastically generated small- and medium-scaled instances (i.e., UACT-ISP instances of scale 3-2-2-4, 5-2-3-4, 10-2-5-4, 10-4-5-4, 20-2-5-4, 20-4-5-8, 30-2-10-4, 30-4-10-8) for training and testing. The convergence curves of the instances at different sizes are given in Figure 7.

To further assess the effectiveness of the models and algorithms presented in this study, the Adaptive Genetic Algorithm (AGA) [11], Genetic Seagull Optimization Algorithm (GSOA) [12], and Non-Dominated Sorting Genetic Algorithm II (NSGA-II) [8] are used as baseline algorithms to solve the models and compare them with our method.

First, three metrics, C_{max} , gap in C_{max} , and runtime, were used for evaluation on stochastically generated small and medium-scaled instances (i.e., scale 3-2-4-2, 5-2-3-4, 10-2-5-4, 10-4-5-4, 20-2-5-4, 20-4-5-8, 30-2-10-4, and 30-4-10-8) (where gap in calculation is shown in Equation (7)); each scale contains 100 instances, and the experimental results are shown in Table 5, with the lowest results for C_{max} and Time highlighted in bold. As can be seen, the trained model consistently outperforms the other heuristics on small- and medium-scaled instances and shows stability as the problem size increases. Although the other three heuristics slightly outperform our method in terms of computation time, this is insufficient when compared to the considerable improvement in the quality of the solution.

Table 5. Results for instances of small and medium sizes. Bold indicates the optimal result.

Size	NSGA-II			GSOA			AGA			OURS		
	C_{max} (s)	Gap (%)	Time (s)	C_{max} (s)	Gap (%)	Time (s)	C_{max} (s)	Gap (%)	Time (s)	C_{max} (s)	Gap (%)	Time (s)
3-2-2-4	145	−2.1	0.013	147	−0.7	0.013	154	3.9	0.018	148	0.00	0.033
5-2-3-4	200	7.5	0.016	213	13.1	0.016	208	11.1	0.025	185	0.00	0.038
10-2-5-4	234	11.5	0.15	249	16.9	0.17	256	19.1	0.29	207	0.00	0.43
10-4-5-4	219	17.3	0.21	227	20.3	0.26	232	22.0	0.37	181	0.00	0.59
20-2-5-4	480	7.7	0.56	499	11.2	0.58	552	20.0	0.67	443	0.00	1.47
20-4-5-8	470	8.5	0.73	479	10.2	0.74	523	17.8	0.98	430	0.00	2.05
30-2-10-4	640	19.5	0.96	640	19.5	0.98	676	23.8	1.30	515	0.00	2.86
30-4-10-8	520	14.8	1.17	527	16.0	1.21	576	23.1	1.56	446	0.00	3.13

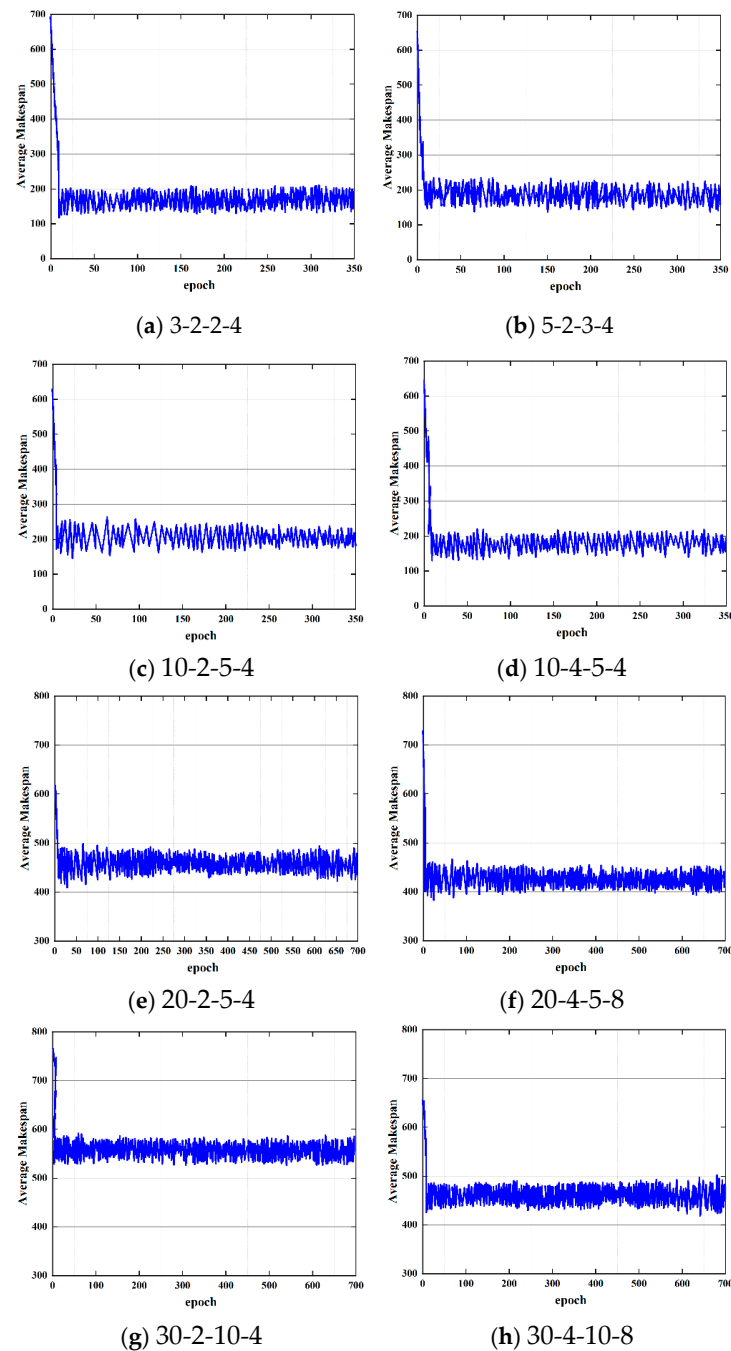


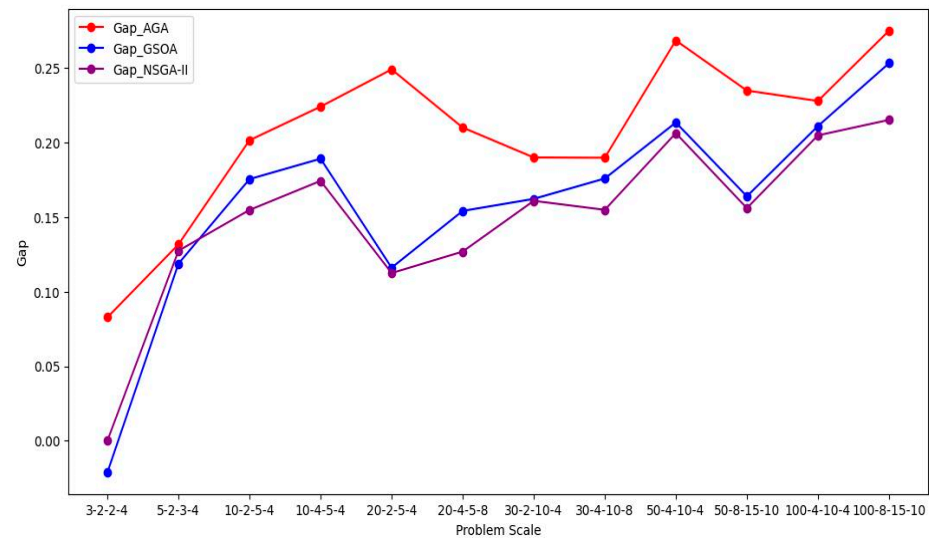
Figure 7. Average makespan convergence curves for training sets of different sizes.

Next, in order to validate the generalizability of the model, four large-scale stochastically generated instances were used for validation. More specifically, we directly used the strategies trained on the instances of 20-2-5-4, 20-4-5-8, 30-2-10-4 and 30-4-10-8 scales to solve the instances of 50-4-10-4, 50-8-15-10, 100-4-10-4 and 100-8-15-10 scales. The set of instances for each scale contains 100 test instances and the computational results are averaged over 100 instances. Table 6 summarizes the results, and we can observe that the models trained using the instances of 20-2-5-4, 20-4-5-8, 30-2-10-4 and 30-4-10-8 scales outperform the other three heuristic algorithms when testing large-scale instances. Thus, the experimental results demonstrate that our method can extract meaningful information from small-scale instances and generalize to large-scale instances.

Table 6. Results for instances of large sizes. Bold indicates the optimal result.

Size	NSGA-II			GSOA			AGA			OURS		
	C_{max} (s)	Gap (%)	Time (s)	C_{max} (s)	Gap (%)	Time (s)	C_{max} (s)	Gap (%)	Time (s)	C_{max} (s)	Gap (%)	Time (s)
50-4-10-4	889	17.9	2.36	897	18.6	2.54	909	19.7	2.78	730	0.00	6.24
50-8-15-10	733	14.7	2.57	746	16.2	2.87	821	23.9	3.12	625	0.00	7.33
100-4-10-4	1780	18.9	6.81	1781	19.0	7.09	1832	21.2	7.62	1444	0.00	15.98
100-8-15-10	1610	21.0	7.08	1695	24.9	7.32	1732	26.5	8.21	1273	0.00	20.19

Figure 8 demonstrates the gap between our method and the other three heuristics on problems of different scales. Based on the method's performance across small-, medium- and large-scale UACT-ISP instances, we conclude that our method can train high-quality scheduling policies with stochastically generated training data, and its generalization performance from training on small-scale UACT-ISP instances to testing on large-scale UACT-ISP instances is promising.

**Figure 8.** Gaps between AGA, GSOA, NSGA-II and our method for problems at different scales.

5.4. Comparative Experiment of Loading and Unloading Modes

Figure 9 represents the process of solving UACT-ISP in the “unloading before loading” mode as a disjunctive model. In this mode, DTQCs must complete all the import operations first, i.e., unloading of imported containers—horizontal transport—stacking of imported containers. All IGVs have to travel empty to the DTQCs to perform the next import task every time they complete an import transport operation, and the DCRCs may be in a vacant state after processing an import container stacking task, waiting for the arrival of the next import task. Until all import tasks are completed, the IGVs will go to perform export tasks, transport the export containers to the DTQCs and then return to the yard empty to transport the next export containers. This mode causes IGVs to incur more idle distances, which leads to longer task durations and affects the makespan. However, in the hybrid mode, at the beginning of the task, DTQC unloads an imported container first, and IGV delivers it to the yard and can then go directly to the export task. Similarly, DTQC and DCRC can also perform import and export tasks alternately. This mode effectively reduces the idle time of the equipment and greatly improves the operational efficiency of the terminal.

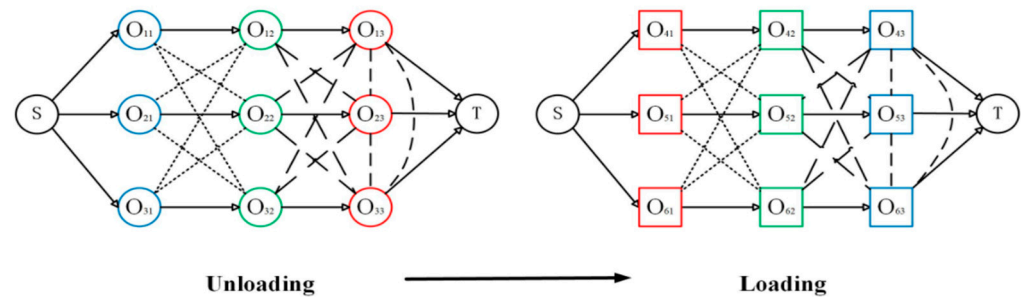


Figure 9. Example of a disjunctive model for UACT-ISP in the unloading before loading mode. All devices prioritize import tasks, with export tasks commencing only after all import tasks are finished.

With the aim of showing the superiority of the hybrid mode, this study evaluates the performance of two different loading and unloading modes on a large-scale instance 100-8-15-10. Each loading and unloading mode is computed 10 times on the instances separately, and the average values of makespan and device spare time are computed. Table 7 shows the comparison results, where the device spare time includes the waiting time of DTQC and DCRC and the time generated by the IGV's unloaded travelling distance. As shown in Table 7, the average of 10 values for operation completion time is 1723.1 s in the “unloading before loading” mode, and the average of 10 values for operation completion time is 1249.5 s in the hybrid mode, which is an increase of 27.5% in efficiency; the average of 10 values for device spare time is 783.4 s and 299.7 s, respectively.

Table 7. Experimental results of different loading and unloading modes under 100-8-15-10.

Number	Unloading Before Loading Mode		Hybrid Mode	
	Makespan	Spare Time	Makespan	Spare Time
1	1657	775	1228	320
2	1731	764	1213	295
3	1711	820	1291	299
4	1678	769	1241	316
5	1780	781	1254	291
6	1766	769	1292	315
7	1769	811	1216	299
8	1664	769	1205	292
9	1727	808	1270	288
10	1748	768	1285	282
Average Value	1723.1	783.4	1249.5	299.7

In summary, selection of the loading and unloading mode can effectively shorten the makespan and device spare time, which helps to enhance the efficiency of terminal operations and device utilization. Further, a comparative analysis of the composition of device spare time under the two loading and unloading modes was carried out, and the results are shown in Figure 10. The device spare time consists of the sum of DTQC spare time, IGV spare time and DCRC spare time. As can be seen from Figure 10, the IGV spare time varies the most under the two loading and unloading modes, indicating that the IGV's load and empty connecting operation is the main factor affecting the efficiency of the whole operation process.

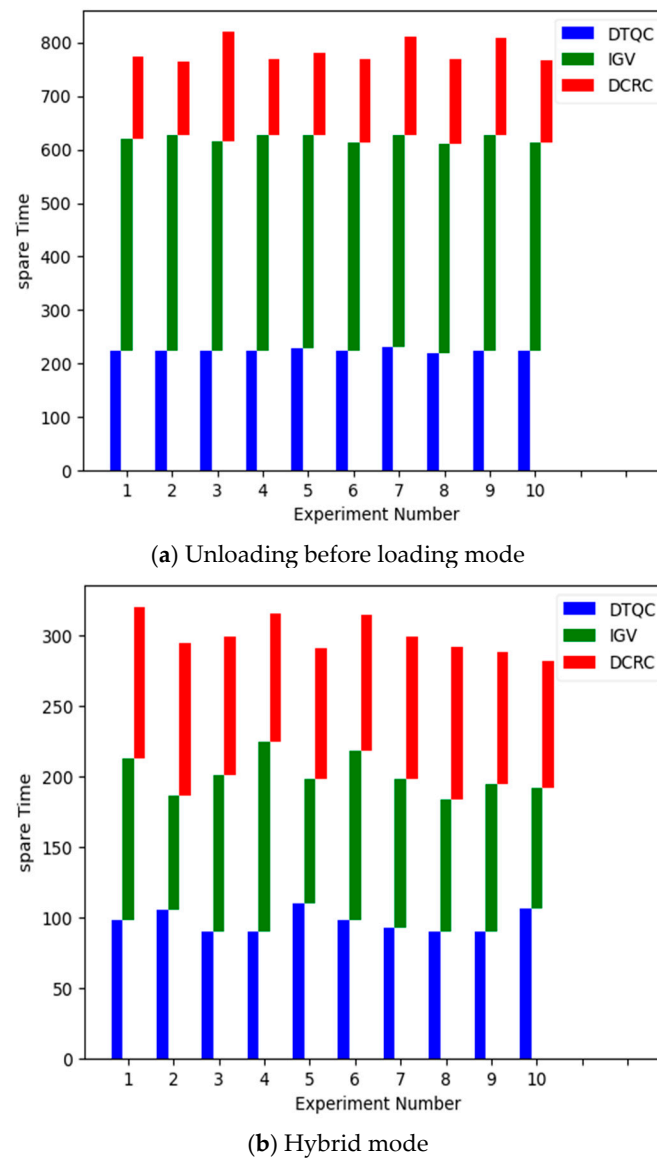


Figure 10. Composition of device spare time in different loading and unloading modes.

5.5. Discussion

Through the above numerical experiments, the following conclusions are derived:

1. The approach presented in this study is superior to the compared heuristic algorithms in terms of computational quality. In practice, the proposed method can optimize the integrated scheduling of DTQC, IGV and DCRC in real time, which reduces makespan and enables more container tasks to be completed in less time. It can significantly improve the operational efficiency of U-shaped automated container terminals.
2. The composition analysis of device spare time reveals that the IGV's spare time varies the most under the two loading and unloading modes. In the "unloading before loading" mode, IGVs have to travel empty to the DTQCs for the next import task after completing an import transport operation, resulting in more idle distances and longer task durations. In contrast, the hybrid mode allows IGVs to directly switch between import and export tasks, reducing their idle time and enhancing the overall operation efficiency. This highlights the importance of optimizing IGV operations in improving the performance of the U-shaped automated container terminal.

This paper focuses primarily on minimizing the makespan as the objective. However, in practical port operations, there are often multiple objectives to consider, such as energy consumption, equipment utilization, and operating costs. Balancing these objectives is essential for achieving sustainable and cost-effective terminal operations. In addition, the proposed method, which combines graph neural networks and deep reinforcement learning, has a relatively high computational complexity. Training the model requires significant computational resources and time, especially for large-scale instances. This may limit its applicability in real-time scheduling scenarios where quick decision-making is necessary. However, with the advancement of computing technology and the development of more efficient algorithms, this issue is expected to be mitigated in the future.

6. Conclusions

This study discusses how the task scheduling of dual trolley quay cranes, intelligent guided vehicles and dual cantilever rail cranes can be integrated in an emerging U-shaped automated container terminal so that all import/export container tasks can be completed in the least amount of time possible. In this study, we treat UACT-ISP as a sequential decision-making problem, innovatively introduce a disjunctive graph model to represent the local states of the problem, and use graph neural networks to embed the local states. Subsequently, we crafted a neural network aimed at learning two distinct strategies: one for job operation actions and another for device actions. These two policies serve to forecast probability distributions of job operations and devices, respectively. Finally, we use the PPO algorithm to train the policy network. In order to verify the superiority of graph neural networks and deep reinforcement learning algorithms for a UACT-ISP, the Non-dominated Sorting Genetic Algorithm (NSGA-II), Adaptive Genetic Algorithm (AGA), and Genetic Seagull Optimization Algorithm (GSOA) are used as baseline methods for comparison with the methods in this study on stochastically generated instances of different scales.

The results show that although the method presented in this study is slightly inferior to the heuristic algorithms in terms of solution speed, it greatly outperforms them in terms of solution quality, especially in the four large-scale problems, which were optimized by 18.1%, 19.7%, and 22.8% on average over NSGA-II, GSOA, and AGA, respectively. To balance the computational quality and speed, we will focus on the improvement of models and algorithms to improve the computational efficiency by lightweighting methods in the future. For example, we can refer to the method in Ref. [32] to address the scalability of hardware demands in actual port deployments from three perspectives: graph construction, semi-supervised learning, and graph clustering.

In addition, this paper conducted 10 experiments on the 100-8-15-10 scale for the “unloading before loading” mode and the hybrid mode, respectively, and evaluated their average makespan and device spare time. The results show that the hybrid mode can enhance the operational effectiveness of the U-shaped ACT by 27.5% compared to the “unloading before loading” mode.

In the future, we can consider introducing multi-objective optimization, such as energy consumption, equipment utilization, and operating costs. By constructing multi-objective optimization models and algorithms, we seek the best balance between different objectives and provide more comprehensive decision support for actual port operations. In addition, the port operating environment is dynamic, with equipment failures, weather changes, vessel delays, and other factors all having an impact on scheduling plans. Future research can further enhance the adaptability of the model to the dynamic environment, and enable the model to perceive environmental changes in real time and dynamically adjust the scheduling strategy by introducing online learning, incremental learning and other mechanisms, so as to improve the robustness and flexibility of the scheduling scheme.

Also, our approach can be used in other layout ACTs to solve more complex scheduling problems, especially in the event that new port equipment may emerge in the future, and new operational characteristics need to be considered.

Author Contributions: Study conception and design: Q.Z., Y.Z. (Yi Zhu); data collection: Y.Z. (Yi Zhu), J.Q., J.D.; analysis and interpretation of results: Y.Z. (Yi Zhu); draft manuscript preparation: Y.Z. (Yi Zhu), Y.Z. (Ying Zhou); formal analysis: H.S.; validation: L.N. All authors have read and agreed to the published version of the manuscript.

Funding: The authors disclosed no financial support for the research, authorship, and/or publication of this article.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

References

- Jonathan, Y.C.E.; Kader, S.B.A. Prospect of Emission Reduction Standard for Sustainable Port Equipment Electrification. *Int. J. Eng.* **2018**, *31*, 1347–1355.
- Yang, Y.; Zhong, M.; Dessouky, Y.; Postolache, O. An integrated scheduling method for AGV routing in automated container terminals. *Comput. Ind. Eng.* **2018**, *126*, 482–493. [[CrossRef](#)]
- Luo, J.; Wu, Y. Scheduling of container-handling equipment during the loading process at an automated container terminal. *Comput. Ind. Eng.* **2020**, *149*, 106848. [[CrossRef](#)]
- Kong, L.; Ji, M.; Yu, A.; Gao, Z. Scheduling of automated guided vehicles for tandem quay cranes in automated container terminals. *Comput. Oper. Res.* **2024**, *163*, 106505. [[CrossRef](#)]
- Zhuang, Z.; Zhang, Z.; Teng, H.; Qin, W.; Fang, H. Optimization for integrated scheduling of intelligent handling equipment with bidirectional flows and limited buffers at automated container terminals. *Comput. Oper. Res.* **2022**, *145*, 105863. [[CrossRef](#)]
- Ji, S.; Luan, D.; Chen, Z.; Guo, D. Integrated scheduling in automated container terminals considering AGV conflict-free routing. *Transp. Lett.* **2021**, *13*, 501–513.
- Wang, Y.; Hu, Z.; Tian, X. Scheduling ASC and AGV considering direct, buffer, and hybrid modes for transferring containers. *Comput. Oper. Res.* **2024**, *161*, 106419. [[CrossRef](#)]
- Duan, J.; Li, L.; Zhang, Q.; Qin, J.; Zhou, Y. Integrated Scheduling of Automatic Guided Vehicles and Automatic Stacking Cranes in Automated Container Terminals Considering Landside Buffer Zone. *Transp. Res. Rec.* **2023**, *2677*, 502–528. [[CrossRef](#)]
- Xu, B.; Jie, D.; Li, J.; Yang, Y.; Wen, F.; Song, H. Integrated scheduling optimization of U-shaped automated container terminal under loading and unloading mode. *Comput. Ind. Eng.* **2021**, *162*, 107695. [[CrossRef](#)]
- Li, J.; Yang, J.; Xu, B.; Yang, Y.; Wen, F.; Song, H. Hybrid Scheduling for Multi-Equipment at U-Shape Trafficked Automated Terminal Based on Chaos Particle Swarm Optimization. *J. Mar. Sci. Eng.* **2021**, *9*, 1080. [[CrossRef](#)]
- Yang, Y.; Sun, S.; Zhong, M.; Feng, J.; Wen, F.; Song, H. A Refined Collaborative Scheduling Method for Multi-Equipment at U-Shaped Automated Container Terminals Based on Rail Crane Process Optimization. *Mar. Sci. Eng.* **2023**, *11*, 605. [[CrossRef](#)]
- Xu, B.; Jie, D.; Li, J.; Zhou, Y.; Wang, H.; Fan, H. A hybrid dynamic method for conflict-free integrated schedule optimization in U-shaped automated container terminals. *J. Mar. Sci. Eng.* **2022**, *10*, 1187. [[CrossRef](#)]
- Niu, Y.; Yu, F.; Yao, H.; Yang, Y. Multi-equipment coordinated scheduling strategy of U-shaped automated container terminal considering energy consumption. *Comput. Ind. Eng.* **2022**, *174*, 108804. [[CrossRef](#)]
- Liu, W.; Zhu, X.; Wang, L.; Wang, S. Multiple equipment scheduling and AGV trajectory generation in U-shaped sea-rail intermodal automated container terminal. *Measurement* **2023**, *206*, 112262. [[CrossRef](#)]
- Cai, L.; Li, W.; Zhou, B.; Li, H.; Yang, Z. Robust multi-equipment scheduling for U-shaped container terminals concerning double-cycling mode and uncertain operation time with cascade effects. *Transp. Res. Part C* **2024**, *158*, 104447. [[CrossRef](#)]
- Jin, X.; Duan, Z.; Song, W.; Li, Q. Container stacking optimization based on Deep Reinforcement Learning. *Eng. Appl. Artif. Intell.* **2023**, *123*, 106508. [[CrossRef](#)]
- Chen, C.; Hu, Z.; Wang, L. Scheduling of AGVs in Automated Container Terminal Based on the Deep Deterministic Policy Gradient (DDPG) Using the Convolutional Neural Network (CNN). *J. Mar. Sci. Eng.* **2021**, *9*, 1439. [[CrossRef](#)]

18. Singh, N.; Akcay, A.; Dang, Q.; Martagan, T.; Adan, I. Dispatching AGVs with battery constraints using deep reinforcement learning. *Comput. Ind. Eng.* **2024**, *187*, 109678. [\[CrossRef\]](#)
19. Zhang, L.; Yan, Y.; Hu, Y. Deep reinforcement learning for dynamic scheduling of energy-efficient automated guided vehicles. *J. Intell. Manuf.* **2023**, *35*, 3875–3888. [\[CrossRef\]](#)
20. Park, J.; Chun, J.; Kim, S.H.; Kim, Y.; Park, J. Learning to schedule job-shop problems: Representation and policy learning using graph neural network and reinforcement learning. *Int. J. Prod. Res.* **2021**, *59*, 3360–3377. [\[CrossRef\]](#)
21. Jing, X.; Yao, X.; Liu, M.; Zhou, J. Multi-agent reinforcement learning based on graph convolutional network for flexible job shop scheduling. *J. Intell. Manuf.* **2022**, *35*, 75–93. [\[CrossRef\]](#)
22. Su, C.; Zhang, C.; Xia, D.; Han, B.; Wang, C.; Chen, G.; Xie, L. Evolution strategies-based optimized graph reinforcement learning for solving dynamic job shop scheduling problem. *Appl. Soft Comput.* **2023**, *145*, 110596. [\[CrossRef\]](#)
23. Lei, K.; Guo, P.; Zhao, W.; Wang, Y.; Qian, L.; Meng, X.; Tang, L. A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. *Expert Syst. Appl.* **2022**, *205*, 117796. [\[CrossRef\]](#)
24. Song, W.; Chen, X.; Li, Q.; Cao, Z. Flexible Job-Shop Scheduling via Graph Neural Network and Deep Reinforcement Learning. *IEEE Trans. Ind. Inform.* **2022**, *19*, 1600–1610. [\[CrossRef\]](#)
25. Huang, J.; Gao, L.; Li, X. An end-to-end deep reinforcement learning method based on graph neural network for distributed job-shop scheduling problem. *Expert Syst. Appl.* **2021**, *238*, 121756. [\[CrossRef\]](#)
26. Zhang, M.; Wang, L.; Qiu, F.; Liu, X. Dynamic scheduling for flexible job shop with insufficient transportation resources via graph neural network and deep reinforcement learning. *Comput. Ind. Eng.* **2023**, *186*, 109718. [\[CrossRef\]](#)
27. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How powerful are graph neural networks? *arXiv* **2018**, arXiv:1810.00826.
28. Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the International Conference on Machine Learning, Lille, France, 6–11 July 2015; pp. 448–456.
29. Zhang, C.; Song, W.; Cao, Z.; Zhang, J.; Tan, P.; Chi, X. Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1621–1632.
30. Nazari, M.; Oroojlooy, A.; Takač, M.; Snyder, L. Reinforcement learning for solving the vehicle routing problem. In Proceedings of the 32nd International Conference on Neural Information Processing Systems, Montreal, QC, Canada, 3–8 December 2018; Volume 31, pp. 9861–9871.
31. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.
32. Daneshfar, F.; Soleymanbaigi, S.; Yamini, P.; Amini, M. A survey on semi-supervised graph clustering. *Eng. Appl. Artif. Intell.* **2024**, *133*, 108215. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.